Selective tool-knowledge adaptation under environment change

PE6203 Group Project 4 | Integrated final report | 30 October 2026

Abstract. We compare no update, full rewrite and dependency-aware selective patching on 18 fixed MCPEvol-Bench tasks across Early, Middle and Late tool environments, using free local models without fine-tuning. The current comparison uses three input-compatible rounds and 486 scored observations. Ten execution exceptions remain included. An earlier incompatible second-round execution is preserved outside the active comparison. Selective patching has the highest mean ECS, 5.913, versus 5.498 for no update and 5.354 for full rewrite, but does not rank first in every round. It uses 65.1% fewer recorded update tokens and 5.5% more recorded completed-grid total tokens than full rewrite. Late TF, recovery and retention remain mixed. We preserve automatic scores, negative cases and the interrupted-attempt overhead. These are descriptive stability repeats, not evidence of statistical significance or universal superiority.

1. Problem and contribution

An enterprise agent can lose capabilities when a tool is renamed, removed, changes its required arguments or adopts different usage rules. Old harness knowledge may then cause invalid calls even though model weights remain unchanged. Updating everything costs computation and may remove useful guidance for tools that did not change.

We implement selective adaptation of tool contracts and usage notes. Public contract differences identify the initial update set; explicit references to affected tool names expand that set. Only impacted usage notes are regenerated. We compare this method against unchanged knowledge and full regeneration through the same actor loop and real archived MCP servers. The contribution is a working, auditable adaptation pipeline and a bounded empirical comparison, not a new foundation model or a benchmark record.

2. System and three policies

Knowledge contains executable tool contracts and usage notes. A contract includes the public name, description and input schema. Server-derived prefixes disambiguate names. Early notes are the original tool descriptions. At an environment transition, added, removed or modified contracts form the initial affected set. Old/current contracts and previous notes are scanned for exact tool-name references; referring tools join the set until closure. Both raw and prefixed names are recognized.

Policy	Contracts after a change	Usage notes after a change
No update	Keep Early contracts	Keep Early notes
Full rewrite	Copy current authoritative contracts	Regenerate all scoped notes
Selective patch	Copy current authoritative contracts	Regenerate affected and explicitly linked notes; preserve the rest

The two updating policies share the model, updater prompt and output validation. The updater receives requested tool names and current contracts, not the task question, reference answer, mutation labels or scores. Exact-name/nonempty-string validation rejects invalid note generations, retains prior knowledge and records consumed tokens. It does not establish semantic correctness. Copying schemas prevents generated notes from redefining parameters; actor planning can still fail.

The dependency graph is lexical. It can miss implicit workflow prerequisites and can expand unnecessarily when a name is only mentioned descriptively. A supplementary component comparison disables this expansion. We do not claim that name/schema checking alone understands operation order or identifier provenance.

The actor permits at most 12 model-decision rounds; one round may contain several tool calls. It receives the task, actual resource listing and policy-specific knowledge, calls current tools and saves raw requests/responses, returns, errors and file changes. A common output-verification opportunity uses an ordinary round. Whole old assistant/tool groups may be pruned under actor context pressure, with pruning recorded. Episode workspaces are isolated; knowledge carries across versions within a task-policy sequence, while task execution state does not. Judge scoring follows execution, preserving a separate actor receipt.

3. Benchmark, models and completed experiments

MCPEvol-Bench is pinned to 888c779a0fe735f612ca62bdd1fdd7c9e6bf8701. Early, Middle and Late use original, iter3 and iter5 server archives. Eleven local MCP packages and dependencies were extracted unchanged; all 32,388 extracted files were compared against the official archives. A Windows import loader supplies file-URL compatibility without modifying tool implementations.

The fixed test IDs are 9, 28, 103, 105, 113, 117, 139, 267, 273, 274, 278, 283, 386, 389, 393, 403, 411 and 416. Development tasks 3 and 101 are disjoint. Selection preceded formal scoring and used local service availability. Sixteen questions concern log-analysis workflows, limiting diversity. Official questions, resources and labels are not rewritten; some requested logs are absent, which is reported rather than repaired with invented data.

Tool scope follows official Early task/server metadata, retaining surviving named tools and newly added tools from those servers in later versions. Exposure means a scoped contract differs from Early, not that the changed tool is indispensable to solving the task.

Environment	Changed / unchanged tasks	Tool / parameter / description change exposure
Middle	9 / 9	3 / 8 / 9 tasks
Late	10 / 8	7 / 10 / 9 tasks

Categories overlap; these counts must not be summed as independent tasks.

Setting	Value
Actor and updater	Qwen3-4B-Instruct-2507 Q4_K_M
Independent local judge	Qwen2.5-7B-Instruct Q4_K_M
Backend / hardware	llama.cpp b11900; RTX 3060 Laptop 6 GB VRAM; 16 GB RAM
Temperature / seed / response limit	0 / 6203 / 2,048 tokens for each model role
Actor/updater context / decision rounds	16,384 tokens / maximum 12
Judge server context	24,576 tokens under the disclosed pre-scoring amendment
Per-round matrix	18 tasks x 3 policies x 3 versions = 162 episodes
Completed main experiments	3 rounds, 486 scored episodes; not 486 distinct tasks
Supplementary work	Original single 6-episode component variant and separate 9-episode development verification; excluded from main repeats

All models run locally; actor/updater and judge are loaded sequentially. API billing is zero, not zero hardware, electricity or development cost. The judge was calibrated on only two development trajectories; that is not general reliability validation.

Round	Scored / expected	Execution exceptions	Analysis status
R1: `final-v1`	162 / 162	4	Included
R2: `final-r2`	162 / 162	1	Included
R3: `final-r3`	162 / 162	5	Included

4. Metrics, comparability and integrity

TF is the original 1-10 task-fulfillment rubric score. PE is Planning Effectiveness in the course brief, represented upstream by planning_and_efficiency_mean; it evaluates redundant/unnecessary actions. ECS is the task-average of each task's mean TF minus its population standard deviation across the three versions. Original prompt formatting, aggregation and ECS functions are retained through a portability adapter for an unavailable import and fixed upstream MCP connection. A valid local score is not a verified task success.

The original second-round run had 123 cross-round resource-list differences and 54 within-round cross-policy mismatches due to Windows path-length-dependent filename visibility. Its 162 scores are preserved, not merged into current results. A replacement run, `final-r2`, was predeclared before execution with the same eight-character directory length as R1 and R3. All 162 no-inference layout checks passed. Its 162 actual resource lists match R1, and all 54 cross-policy input groups agree. R3 previously passed its actual-input checks. The active comparison therefore contains R1, corrected R2 and R3: 486 repeated observations over 18 distinct tasks, with three observations per task/policy/version. Inclusion is based on inputs and declared conditions, not favorable scores. The corrected R2 selective ECS is lower than both baselines and remains included.

Two historical differences remain disclosed. The first 39 original judgments used a 16,384-token client guard, later ones 24,576; the server was always 24,576. All 39 fit the earlier guard (largest reserved estimate 15,505), and all 162 original judge calls retain zero dropped messages. One original public ZIP result differs from its local frozen counterpart in JSON serialization while values and types agree. Both source hashes and a precise copy are retained. We do not claim every client implementation or serialized byte is identical.

Fixed seed and temperature-zero repetitions describe execution stability under the same settings, not independent new tasks or random sampling. Early has identical initial knowledge and no updating, yet its separate executions already differ. This weakens causal attribution of later policy gaps. All 10 active-grid execution exceptions remain included; completed actor episodes and completed scores are not retried to improve results. Score reconstruction uses only existing events and preserves failure originals.

5. Results

5.1 Three comparable rounds

R1 final-v1

Policy	Early TF / PE	Middle TF / PE	Late TF / PE	ECS
No update	6.83 / 6.56	7.28 / 7.33	7.44 / 7.89	6.280
Full rewrite	6.28 / 7.50	7.06 / 7.83	6.72 / 7.17	5.239
Selective patch	7.11 / 7.33	7.61 / 7.61	7.17 / 7.39	6.332

R2 final-r2

Policy	Early TF / PE	Middle TF / PE	Late TF / PE	ECS
No update	7.17 / 7.39	5.61 / 7.06	7.78 / 7.72	5.285
Full rewrite	6.44 / 6.83	6.33 / 6.94	7.94 / 8.56	5.484
Selective patch	7.00 / 8.06	6.78 / 6.94	6.17 / 6.94	5.255

R3 final-r3

Policy	Early TF / PE	Middle TF / PE	Late TF / PE	ECS
No update	6.06 / 6.94	7.50 / 7.22	6.78 / 6.61	4.929
Full rewrite	6.89 / 7.28	6.61 / 7.11	7.56 / 8.00	5.340
Selective patch	7.72 / 7.22	7.06 / 7.39	6.56 / 7.17	6.152

5.2 Current three-round comparison

Policy	Early TF / PE	Middle TF / PE	Late TF / PE	ECS mean +/- SD
No update	6.69 / 6.96	6.80 / 7.20	7.33 / 7.41	5.498 +/- 0.701
Full rewrite	6.54 / 7.20	6.67 / 7.30	7.41 / 7.91	5.354 +/- 0.123
Selective patch	7.28 / 7.54	7.15 / 7.31	6.63 / 7.17	5.913 +/- 0.577

SD is the sample standard deviation of three round means (n=3), not a confidence interval or the SD over tasks. Each stage mean contains 54 repeated observations per policy and version, from 18 distinct tasks. Selective patch has the highest pooled ECS mean but the lowest ECS in corrected R2 (5.255 versus full rewrite 5.484 and no update 5.285). Its pooled Late TF/PE is 6.63/7.17, below full rewrite 7.41/7.91. Early differences already exist before adaptation. These results do not isolate a causal policy advantage.

5.3 Recovery and retention

Changed-scope recovery is TF minus same-version no update; unchanged-scope retention is TF minus that policy's own Early score. Both are paired within task and round. Categories and task counts vary by environment.

Policy	Version	Cohort	Tasks / observations	TF	Recovery or retention
No update	Middle	changed	9 / 27	7.48	+0.00
No update	Middle	unaffected	9 / 27	6.11	-0.37
No update	Late	changed	10 / 30	7.67	+0.00
No update	Late	unaffected	8 / 24	6.92	+0.88
Full rewrite	Middle	changed	9 / 27	7.56	+0.07
Full rewrite	Middle	unaffected	9 / 27	5.78	+0.04
Full rewrite	Late	changed	10 / 30	7.83	+0.17

Policy	Version	Cohort	Tasks / observations	TF	Recovery or retention
Full rewrite	Late	unaffected	8 / 24	6.88	+1.67
Selective patch	Middle	changed	9 / 27	7.44	-0.04
Selective patch	Middle	unaffected	9 / 27	6.85	-0.37
Selective patch	Late	changed	10 / 30	7.13	-0.53
Selective patch	Late	unaffected	8 / 24	6.00	-1.04

Selective changed-task recovery is -0.04 in Middle and -0.53 in Late; its unchanged-task retention is -0.37 and -1.04. Full rewrite recovery is +0.07/+0.17 and retention +0.04/+1.67. This comparison does not establish a retention or recovery advantage for selective patch.

6. Update cost and total recorded cost

The table sums returned and saved calls from the three completed grids. Seconds are cumulative request durations, not elapsed experiment time.

Policy	Phase	Calls	Tokens	Request seconds
No update	update	0	0	0.00
No update	agent	1285	9,133,062	37234.02
No update	judge	162	1,242,483	4203.26
Full rewrite	update	108	313,370	932.83
Full rewrite	agent	1214	8,199,730	19041.92
Full rewrite	judge	162	1,209,904	5638.78
Selective patch	update	57	109,347	336.24
Selective patch	agent	1232	8,948,412	12700.07
Selective patch	judge	162	1,197,677	6588.86

Selective patch uses 109,347 update tokens versus 313,370, a 65.1% reduction. Its completed-grid total is 10,255,436 tokens versus 9,723,004, 5.5% higher. Update savings are not total-token or total-compute savings.

Corrected R2 experienced a monitoring MemoryError after 156 records were committed. Those 156 records were preserved. The entire uncommitted task 416/full rewrite/Early attempt was archived, the inference process was restarted with unchanged settings, and only that unfinished episode was restarted before executing the other five episodes. No completed episode or score was retried to improve a result. The partial attempt adds seven returned calls, 75,652 recorded tokens and 295.61 request seconds outside the grid totals above, attributable to full rewrite. Including that known overhead gives full rewrite 9,798,656 tokens and selective patch 10,255,436 tokens: selective remains about 4.7% higher. Unreturned usage is unknown, so this is not complete end-to-end cost accounting.

Local API billing is zero; hardware, electricity and development effort are not monetized. Original judge allocation interruptions and unknown unreturned costs remain disclosed.

7. Failure analysis and judge disagreement

7.1 Under-update: required parameters, task 113

Early planning requires originalRequest and tasks; Middle adds splitDetails; Late also requires priority and tags. In R3 no-update Late, the first planning call omits the new fields. Six further planning calls supply splitDetails but still omit priority and tags. All seven fail required-field validation; the root planning request is not registered. An empty project task store is not evidence of a successful plan. This is direct stale-parameter evidence and the no-update limiting case of under-update, not proof that selective patch omitted those fields.

R3 full rewrite and selective patch Late supply all five required fields and save plans. The selective actor also supplies estimatedDeadline. Saved tasks remain unfinished/unapproved; this establishes plan registration, not implemented dashboard code, tests or genuine human approval. R3 no-update Late nevertheless receives TF/PE 10/10, while selective Middle/Late receive TF 4. Original scores are preserved, so successful registration cannot be equated with measured TF recovery or dashboard completion.

7.2 Over-update candidate: unchanged guidance, task 28

The task compares two logs and requires saved to-do items. R3 full rewrite Middle has no scoped contract changes but regenerates all three notes. It then invents source_id_1 for task addition and tries to initialize two .txt logs as task sources, producing one missing-source and two unsupported-format errors. No target task file is created. Selective Middle preserves the notes and writes tasks through a valid initialization/addition sequence.

R3 full rewrite Late again rewrites all notes despite only the reader contract changing, and fails to save the target tasks. Selective Late recovers from one parameter error and saves a task file after four calls. These observations associate unnecessary rewriting with regression, but do not isolate a sentence-level cause: the actor also disobeys guidance. The original round contains a related over-update candidate and counterexamples. Some successfully written tasks overstate an unresolved database fault absent from today's log; file existence alone does not prove grounded content.

7.3 Other errors and misleading scores

R3 task 117 selective Early stops with an exception, an empty answer and no task file, yet receives 10/10. It occurs before any updating, so cannot be a selective-update failure. Full rewrite Middle has three split_tasks returns explicitly describing invalid JSON while isError is false, no saved Shrimp tasks, and also receives 10/10. Full rewrite Late fails at its third call and receives TF/PE 8/10. Flagged-error counts therefore miss semantic errors, and high scores do not establish actual completion.

R3 selective Middle task 117 does save six tasks, but then uses a task ID as another tool system's source_id; both history searches fail. Identifier types can match while identifier provenance and workflow ordering are wrong. A successful empty query of work.md in no-update Late also does not mean reusable historical experience was found. No task storage, no successful search, and a normal empty result are distinct outcomes.

R3 execution exception	Task	TF / PE
Full rewrite Late	117	8 / 10
No update Middle	389	7 / 4
Selective patch Late	416	7 / 7
Selective patch Early	117	10 / 10
Selective patch Early	389	8 / 4

All five have empty final answers and remain included. Generic infrastructure_error/ExceptionGroup labels do not establish hardware root causes. We separate stale knowledge, unrelated-note changes, actor planning/ID

errors, serialization/tool faults, absent resources and judge disagreement. We do not replace automated scores with favorable manual labels or discard failures. Concrete calls, returned text and saved artifacts support each interpretation.

7.4 Supplementary dependency-expansion comparison

The original single component variant disables dependency expansion on tasks 113 and 416 across three versions, adding six episodes outside main repeats. The original expanded method is lower by 6 TF points on task 113 Middle and task 416 Late, tied on two other changed-stage pairs; an Early difference also occurs before any expansion. This small, single comparison does not establish expansion's benefit or a controlled causal disadvantage. It was not repeated, and is not included among the 486 main scores.

8. Discussion and limits

Selective updating can reduce note-generation work and avoid rewriting unaffected knowledge. It is most plausible when public changes are localized and usefully described by explicit dependencies. It can fail when usage rules are implicit, generated notes are incomplete, the actor violates correct guidance, resources are unavailable or the judge rewards unsupported plans. No-update can remain competitive when updated tools are not needed, or measured scores fail to detect actual errors.

Our quality evidence is limited by 18 tasks dominated by log workflows, small quantized models, three comparable fixed-setting repeats and a fallible local judge. Early imbalance, output paths, generated IDs and numeric implementation behavior prevent treating score gaps as isolated update effects. Timing is machine/order/cache dependent. We claim neither significance nor state-of-the-art performance. More diverse tasks, independently designed repeats, grounded outcome checks and genuine human assessment would be needed for stronger conclusions.

The engineering lesson is to check valid calls, correctly sourced IDs, prerequisites and persisted outputs after a tool update. A fluent answer, a successful MCP flag or a high judge score alone is insufficient. Lower update cost and recovery/retention should be assessed separately.

9. Reproduction evidence and team responsibilities

The reproduction guide documents setup and inspection. The evidence index separates the current R1/corrected-R2/R3 selection from the superseded incompatible execution. The corrected second-round archive is included locally in the repository, with manifest hashes, all 162 results and actor copies, event logs, changed outputs, the original failed record, and the archived interrupted attempt. R1 and R3 retain their byte-verified published originals. Historical dated receipts remain snapshots and are not relabeled as current checks.

Code development, report preparation and evidence checks used AI assistance. The README lists the six members and their responsibility areas. File ownership and uploading do not establish independent authorship or independent human assessment.

10. Conclusion

We built and evaluated selective tool-knowledge adaptation using unchanged local models and evolving MCP tools. Three comparable rounds contain 486 scored observations on 18 distinct tasks. Selective patch reduces update tokens by 65.1% and has the highest pooled ECS, but corrected R2 favors full rewrite. Completed-grid total tokens rise by 5.5%, and recovery and retention do not consistently improve. Preserved failures and judge disagreements support a bounded efficiency finding, not universal quality or total-cost superiority.

Evidence and sources

Course brief: provided PE6203 Project 4. Sources: pinned benchmark, current metrics, 486 source scores and hashes, R2 cases, R3 cases, and archive checksums.